Lean4-Automata: Technical Architecture Reference

Documentation Anchor — Commit `10927ba`

Auto-generated from codebase survey

May 2026

Abstract

This document is the *architecture anchor* for the Lean4-Automata project at commit `10927ba` (HEAD of `main` as of May 2026). It captures every database column, HTTP route, environment variable, Go flow-function, and Docker service so that future checkpoint updates can be expressed as deltas against this baseline. The system indexes Lean 4 proof libraries into a PostgreSQL database, exposes them via dual FastAPI micro-services, and provides a Go-based forum harness for AI-agent consumption of proof-state intelligence.

Contents

1 Introduction **3**
- 1.1 Purpose . . . 3
- 1.2 Design Goals . . . 3
- 1.3 Scope of This Document . . . 3

2 System Overview **4**
- 2.1 Key Technology Choices . . . 4

3 Database Schema **5**
- 3.1 Table: `projects` . . . 5
- 3.2 Table: `files` . . . 5
- 3.3 Table: `symbols` . . . 6
- 3.4 Table: `dependencies` . . . 6
- 3.5 Table: `file_dependencies` . . . 7
- 3.6 Table: `index_runs` . . . 7
- 3.7 Views (Migration 0004) . . . 7
- 3.8 Materialized Views (Migration 0003) . . . 7

4 Parser Static-Analysis Signals **8**
- 4.1 `FileMetrics` Fields . . . 8
- 4.2 `ParsedSymbol` Fields . . . 8
- 4.3 Parse Pipeline . . . 9

5 Python API Routes **10**
- 5.1 Query API (`:8000`) . . . 10
- 5.2 Ingestion API (`:8001`) . . . 10
- 5.3 Celery Workers . . . 10

6 Go Forum Harness **11**
6.1 HTTP Routes . . . 11
6.2 `FlowFunc` Registry . . . 11
6.3 SQLite Forum Schema . . . 12
6.4 Security Posture . . . 12

7 Infrastructure Services **13**
7.1 Service Matrix . . . 13
7.2 Nginx Reverse Proxy . . . 13
7.3 Volumes . . . 13

8 Configuration Reference **14**
8.1 Application & Ports . . . 14
8.2 Database . . . 14
8.3 Redis & Celery . . . 14
8.4 JWT Authentication . . . 14
8.5 Runtime . . . 14
8.6 Parser & Watcher . . . 15
8.7 Go Forum Harness . . . 15
8.8 Grafana . . . 15

9 Observability Stack **16**
9.1 Loki (Log Aggregation) . . . 16
9.2 Grafana (Dashboards) . . . 16
9.3 Celery Flower . . . 16
9.4 Application-Level Metrics . . . 16

10 Future Work **18**
10.1 Not-Yet-Implemented Flows . . . 18
10.2 CLI Wizard . . . 18
10.3 Micro-Agent Harness . . . 18
10.4 Formula-to-Lean Matcher . . . 18
10.5 Additional Planned Enhancements . . . 19

11 Commit History **20**

1 Introduction

1.1 Purpose

Lean4-Automata is a full-stack indexing, search, and analysis platform for Lean 4 proof libraries. It continuously parses `.lean` source files, extracts every top-level declaration (theorems, definitions, axioms, instances, etc.), computes static-analysis quality signals, and stores the results in a PostgreSQL database fronted by dual FastAPI micro-services (read-only query API and admin-only ingestion API). A Next.js 16 frontend and a Go-based *forum harness* provide two complementary consumption paths: a rich browser UI for human analysts and an SSE-driven flow-execution engine for AI agents (TPH, DynamicFlux, Wolfram) that need machine-readable proof-state intelligence.

1.2 Design Goals

1. **AI-agent consumption.** Every endpoint returns structured JSON with pagination, full-text-search ranking, and dependency-graph traversals so that large-language-model agents can reason about proof completeness without re-parsing source files.

2. **Incremental indexing.** A file-watcher daemon debounces filesystem events, gates on SHA-256 hashes, and upserts only changed files—keeping the index fresh with sub-second latency.

3. **Static proof-quality signals.** The parser emits 15+ `FileMetrics` fields (`sorry_count`, `proof_completion`, `native_decide_count`, etc.) that enable proof-health dashboards without elaboration-level tooling.

4. **Local-first forum harness.** The Go forum stores all findings in a local SQLite WAL database (`~/.lean4a/forum.db`), ensuring that sensitive proof state never leaves the user's machine unless explicitly exported.

5. **Observable infrastructure.** Docker Compose orchestrates 10 services with health checks, rate limiting, and an optional Loki + Grafana observability stack.

1.3 Scope of This Document

This document is the *architecture anchor* at commit `10927ba`. It captures every schema column, every HTTP route, every environment variable, and the full flow-function registry so that subsequent checkpoint documents can be expressed as deltas against this baseline.

2 System Overview

Figure 1 shows the six-layer architecture. Data flows bottom-to-top: the *Parser* layer watches the filesystem and feeds PostgreSQL; the *API* layer exposes read/write endpoints; the *Forum Harness* orchestrates CLI flows via SSE; and the *UI* and *TPH/Agent* layers consume the results.

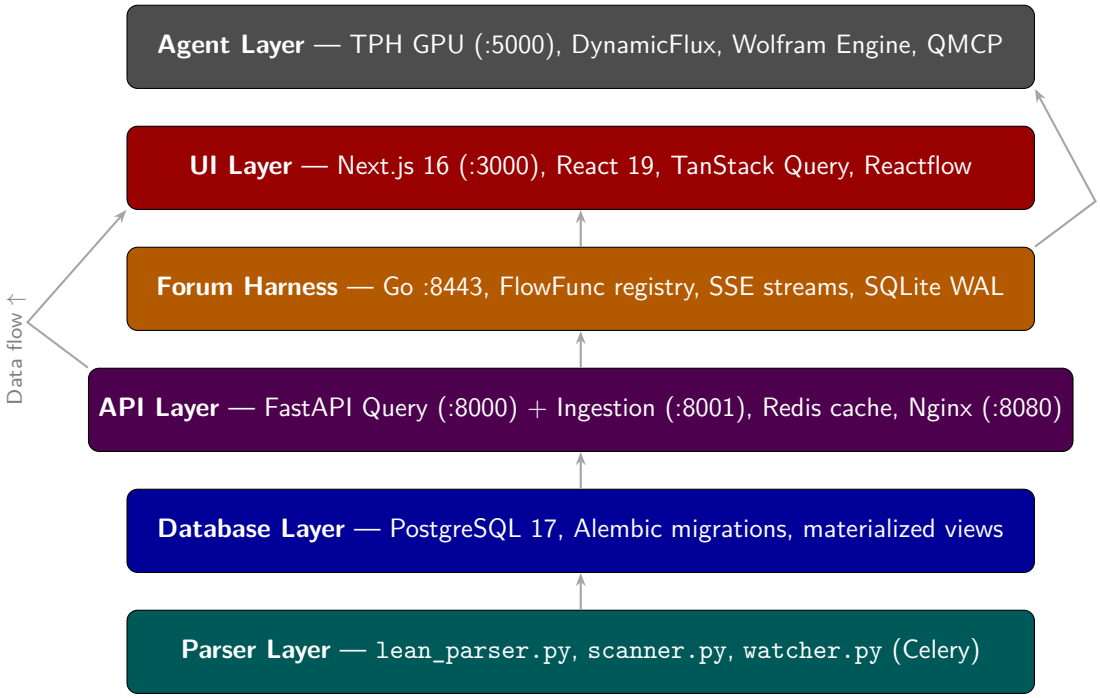


Figure 1: Six-layer architecture of Lean4-Automata at commit `10927ba`.

2.1 Key Technology Choices

Layer	Technology	Rationale
Parser	Python 3.11, regex-based	Fast; no Lean toolchain dependency
Database	PostgreSQL 17, JSONB, GIN	Rich indexing, CTE support
Cache	Redis 7, pub/sub	Cache-aside + invalidation bus
Query API	FastAPI + Uvicorn	Async, OpenAPI, SSE native
Ingestion API	FastAPI + Celery	Async task queue for long scans
Forum Harness	Go 1.21+, SQLite WAL	Low-latency subprocess control
Frontend	Next.js 16, React 19	App Router, SSR, TanStack Query
Observability	Loki 3.4 + Grafana 11.6	Structured log aggregation

3 Database Schema

The PostgreSQL schema is managed by Alembic (async driver `asyncpg`). The migration chain is:

Rev	File	Description	Objects Created
0001	`0001_initial_schema.py`	Core tables + enums	6 tables, 2 enums
0002	`0002_add_indexes.py`	B-tree + GIN indexes	20+ indexes
0003	`0003_materialized_views.py`	Centrality + module metrics	2 mat. views
0004	`0004_enriched_metrics.py`	Phase 2 static-analysis	15 columns, 3 views

3.1 Table: `projects`

Column	Type	Notes
id	`INTEGER` PK	Auto-increment
name	`VARCHAR(255)`	`NOT NULL, UNIQUE`
root_path	`TEXT`	Filesystem root
lean_version	`VARCHAR(50)`	e.g. `"4.8.0"`
lakefile_hash	`VARCHAR(64)`	Lake config hash
indexed_at	`TIMESTAMPTZ`	Default `now()`
last_modified	`TIMESTAMPTZ`	Default `now()`
metadata	`JSONB`	Free-form metadata

3.2 Table: `files`

Column	Type	Notes
id	`INTEGER` PK	Auto-increment
project_id	`INTEGER` FK	`projects.id`, CASCADE
relative_path	`TEXT`	Path relative to project root
absolute_path	`TEXT`	Full filesystem path
module_name	`TEXT`	Lean module name
file_hash	`VARCHAR(64)`	SHA-256 (hash gate)
line_count	`INTEGER`	Number of lines
char_count	`INTEGER`	Number of characters
indexed_at	`TIMESTAMPTZ`	Default `now()`
last_modified	`TIMESTAMPTZ`	Default `now()`
imports	`JSONB`	JSON array of import strings
metadata	`JSONB`	Free-form metadata
Phase 2 enriched columns (migration 0004):		
sorry_count	`INTEGER`	Default 0. Raw `sorry` occurrences
admit_count	`INTEGER`	Default 0. Raw `admit` occurrences
sorry_lines	`JSONB`	Default `[]`. 1-based line numbers
sorry_in_proof_count	`INTEGER`	Default 0. Theorems with sorry in body
theorem_count	`INTEGER`	Default 0. Total theorems + lemmas
proof_completion	`FLOAT`	Default 1.0. Ratio of complete proofs
native_decide_count	`INTEGER`	Default 0. Fragile kernel decisions
debug_command_count	`INTEGER`	Default 0. Leftover `#check`/`#eval`

Column	Type	Notes
noncomputable_count	INTEGER	Default 0. `noncomputable` decls
instance_count	INTEGER	Default 0. Typeclass instances
declared_axiom_count	INTEGER	Default 0. Custom axiom decls
simp_attr_count	INTEGER	Default 0. `@[simp]` tagged lemmas
docstring_count	INTEGER	Default 0. Doc comments present
todo_count	INTEGER	Default 0. TODO/FIXME/HACK/XXX
mathlib_import_count	INTEGER	Default 0. Imports from `Mathlib.*`

Unique constraint: `(project_id, relative_path)`.

3.3 Table: `symbols`

Column	Type	Notes
id	INTEGER PK	Auto-increment
file_id	INTEGER FK	`files.id`, CASCADE
project_id	INTEGER FK	`projects.id`, CASCADE
name	TEXT	Short name (`"append"`)
full_name	TEXT	Qualified name (`"List.append"`)
kind	`symbol_kind`	Enum (see below)
line_start	INTEGER	1-based start line
line_end	INTEGER	Inclusive end line
col_start	INTEGER	1-based column start
col_end	INTEGER	1-based column end
signature	TEXT	Lean signature text
docstring	TEXT	`/- ... -/` doc comment
is_public	BOOLEAN	Default `true`
is_deprecated	BOOLEAN	Default `false`
indexed_at	TIMESTAMPTZ	Default `now()`
attributes	JSONB	e.g. `["@[simp]"]`
metadata	JSONB	Free-form
has_sorry	BOOLEAN	Default `false` (Phase 2)

Unique constraint: `(project_id, full_name)`.
`symbol_kind` enum values: `theorem`, `def`, `axiom`, `instance`, `inductive`, `structure`, `class`, `constant`, `opaque`, `abbrev`.

3.4 Table: `dependencies`

Column	Type	Notes
id	INTEGER PK	Auto-increment
source_symbol_id	INTEGER FK	`symbols.id`, CASCADE
target_symbol_id	INTEGER FK	`symbols.id`, CASCADE
dependency_type	`dep_type`	Enum: `direct`, `implicit`, `import`, `indirect`
confidence	FLOAT	Default 1.0 (0.0–1.0)

indexed_at	TIMESTAMPTZ	Default now()
provenance	TEXT	Detection method
metadata	JSONB	Free-form

3.5 Table: file_dependencies

Column	Type	Notes
id	INTEGER PK	Auto-increment
source_file_id	INTEGER FK	files.id, CASCADE
target_file_id	INTEGER FK	files.id, CASCADE
is_direct_import	BOOLEAN	Default true
indexed_at	TIMESTAMPTZ	Default now()

3.6 Table: index_runs

Column	Type	Notes
id	INTEGER PK	Auto-increment
project_id	INTEGER FK	projects.id, CASCADE
run_type	VARCHAR(50)	"full_scan" or "incremental"
started_at	TIMESTAMPTZ	Default now()
completed_at	TIMESTAMPTZ	Nullable
files_processed	INTEGER	Default 0
symbols_indexed	INTEGER	Default 0
errors	JSONB	Default []
status	VARCHAR(50)	"running", "success", "failed"

3.7 Views (Migration 0004)

v_proof_status Per-file proof completeness. Key columns: relative_path, theorem_count, sorry_in_proof_count, proof_completion, proof_status (derived: complete, partial, mostly_incomplete, no_theorems).

v_floating_symbols Public theorems/defs with zero incoming cross-file dependencies—proved but never referenced.

v_file_interconnect File-level fan-in/fan-out, mathlib_import_count, local_import_count, and per-file health summary.

3.8 Materialized Views (Migration 0003)

dependency_stats Per-symbol centrality: outgoing_dependencies, incoming_dependencies, total_connections, fan_out_ratio.

module_metrics Per-module aggregations: symbol counts by kind, avg_symbol_length_lines, documentation_coverage_pct.

4 Parser Static-Analysis Signals

The regex-based parser (`parser/lean_parser.py`) extracts top-level declarations without requiring a Lean 4 elaborator. For each `.lean` file it produces a `FileMetrics` frozen dataclass whose fields map 1:1 to columns on the `files` table (added in migration 0004).

4.1 `FileMetrics` Fields

Field	Type	Description	SQL Column
Proof completeness			
sorry_count	`int`	Raw `sorry` occurrences	`sorry_count`
admit_count	`int`	Raw `admit` occurrences	`admit_count`
sorry_lines	`list[int]`	1-based line numbers of sorrys	`sorry_lines`
sorry_in_proof_count	`int`	Theorems whose body contains sorry	`sorry_in_proof_count`
theorem_count	`int`	Total theorems + lemmas	`theorem_count`
proof_completion	`float`	$\frac{thm_count - sorry_in_proof}{thm_count}$	`proof_completion`
Fragility signals			
native_decide_count	`int`	`native_decide` (kernel-level, fragile)	`native_decide_count`
debug_command_count	`int`	Leftover `#check`/`#eval`/`#print`	`debug_command_count`
Symbol counts			
noncomputable_count	`int`	`noncomputable` declarations	`noncomputable_count`
instance_count	`int`	Typeclass instances	`instance_count`
declared_axiom_count	`int`	Custom `axiom` declarations	`declared_axiom_count`
simp_attr_count	`int`	`@[simp]`-tagged lemmas	`simp_attr_count`
Documentation / hygiene			
docstring_count	`int`	`/- ... -/` docstrings	`docstring_count`
todo_count	`int`	TODO / FIXME / HACK / XXX comments	`todo_count`
Import profile			
mathlib_import_count	`int`	Imports from `Mathlib.*`	`mathlib_import_count`

4.2 `ParsedSymbol` Fields

Each top-level Lean declaration yields a `ParsedSymbol`:

Field	Type	Description
name	`str`	Short name (e.g. `"append"`)
kind	`str`	Kind keyword (`def`, `theorem`, `lemma`, ...)
line_start	`int`	1-based line number
namespace_path	`str`	Namespace context (e.g. `"List"`)

is_private	`bool`	Whether marked `private`
signature_text	`str`	Lean signature snippet
mods	`str`	Raw modifier string (e.g. `"@[simp] noncomputable"`)
has_sorry	`bool`	`sorry`/`admit` found in declaration body

Kind mapping: The parser recognises 11 keywords (`def`, `theorem`, `lemma`, `instance`, `class`, `structure`, `inductive`, `abbrev`, `axiom`, `example`, `opaque`). At the DB layer `lemma` → `theorem` (collapsed) and `example` is dropped.

4.3 Parse Pipeline

1. Strip single-line (`--`) and block (`/- ...-/`) comments, preserving line numbers.
2. Extract imports: `r"^\s*import\s+([A-Za-z0-9_.]+)\s*$"`
3. Track `namespace`/`end` blocks for fully-qualified names.
4. Match declaration heads via modifier + kind + name regex.
5. Scan for `sorry`, `admit`, `native_decide`, `#check`, `#eval`, `@[simp]`, etc.
6. Aggregate into `FileMetrics`.

Constraint: Not a full Lean 4 grammar—regex extraction of declaration *heads* only; tactic blocks and elaborator macros are out of scope.

5 Python API Routes

Two FastAPI applications expose the database to consumers.

5.1 Query API (:8000)

Base prefix: `/api/v1`. CORS allowed from `http://localhost:3000`. No authentication required.

Method / Path	Handler	Description
`GET /health/live`	liveness	Returns `{"status":"alive"}`
`GET /health/ready`	readiness	Checks DB + Redis; returns booleans
`GET /api/v1/symbols/search`	`search_symbols`	Full-text search. Params: `q`, `project_id`, `limit` (1–100), `offset`
`GET /api/v1/symbols/{full_name}`	`get_symbol`	Lookup by qualified name
`GET /api/v1/dependencies/graph`	`get_dep_graph`	Transitive + reverse deps. Params: `full_name`, `max_depth` (1–20)
`GET /api/v1/modules/metrics`	`get_module_metrics`	Module-level aggregations. Params: `project_id`, `order_by`
`GET /api/v1/events/invalidation`	SSE stream	`text/event-stream`; publishes cache-invalidation events

Cache strategy. Redis cache-aside with 1-hour TTL. Keys follow the pattern `lean4:cache:{entity}:{params` Invalidation is broadcast via Redis pub/sub channel `lean4:invalidate`.

5.2 Ingestion API (:8001)

Base prefix: `/admin`. Requires Bearer JWT with `role:"admin"`.

Method / Path	Handler	Description
`POST /health/live`	liveness	Returns `{"status":"alive"}`
`POST /health/ready`	readiness	DB + Redis check (503 if unhealthy)
`POST /admin/index/start`	`start_index`	Enqueue Celery full-scan. Body: `project_id`, `root_dir`. Returns 202 with `task_id`
`GET /admin/index/status/{task_id}`	`index_status`	Poll Celery task state

5.3 Celery Workers

Task	Description
`full_scan_task`	Parse all `.lean` files under `root_dir`, upsert to DB
`refresh_views_task`	Refresh materialized views (`dependency_stats`, `module_metrics`). Scheduled via Celery Beat

6 Go Forum Harness

The forum harness is a Go HTTP server (`go/forum.go`, port 8443) that extends the existing Go_Red QMCP consent gate with a flow-execution engine, SSE log streaming, ticket storage, and proxy endpoints for the Lean4-Automata and TPH APIs.

6.1 HTTP Routes

Method / Path	Handler	Description
`GET /flows`	`handleListFlows`	List all flow definitions
`POST /flows/{name}/start`	`handleStartFlow`	Execute a named flow
`GET /flows/{name}/events`	SSE stream	Real-time execution logs
`GET /tickets/{id}`	`handleGetTicket`	Retrieve ticket by ID
`GET /tickets`	`handleListTickets`	List recent tickets (paginated)
`POST /sessions`	`handleCreateSession`	Create a new session
`GET /sessions`	`handleListSessions`	List sessions
`GET /sessions/{id}`	`handleGetSession`	Session details + tickets
`POST /lean/query`	proxy	Forward to Lean4-Automata query API
`POST /tph/inference`	proxy	Forward to TPH GPU server
`GET /env/status`	`handleEnvStatus`	Python env + GPU check
`GET /health`	health	DB + upstream health

6.2 `FlowFunc` Registry

All flows are compiled into the binary—no runtime code execution. The `CommandTemplate` field (present in early designs) was eliminated in commit `10927ba`.

```
type FlowFunc func(
    ctx      context.Context,
    params   map[string]interface{},
    w        http.ResponseWriter,
    flusher http.Flusher,
) (json.RawMessage, string, string)
```

Listing 1: FlowFunc type signature

Flow Name	Handler	Type	Endpoint / Notes
`lean_sorry_scan`	`flowLeanSorryScan`	builtin	`/api/v1/` query
`lean_float_scan`	`flowLeanFloatScan`	builtin	`v_floating_symbols`
`lean_file_interconnect`	`flowLeanFileInterconnect`	builtin	`v_file_interconnect`
`lean_db_health`	`flowLeanDBHealth`	builtin	DB stats query
`quick_proof_report`	`flowQuickProofReport`	builtin	Composite report
`tph_inference`	`flowTPHInference`	proxy	`:5000/inference`
`tph_status`	`flowTPHStatus`	proxy	`:5000/status`
`env_check`	`flowEnvCheck`	builtin	Python env probe
`gpu_check`	`flowGPUCheck`	builtin	ROCm / AMD GPU
`formula_match`	placeholder	—	*Not implemented*

Flow Name	Handler	Type	Endpoint / Notes
`wolfram_verify`	placeholder	—	*Not implemented*
`lean_diff_mathlib`	placeholder	—	*Not implemented*

6.3 SQLite Forum Schema

Path: `~/.lean4a/forum.db` (WAL mode). Four tables:

`tickets` (`id TEXT PK`): Flow results. Columns: `flow_name`, `input_json`, `output_json`, `status` (`CHECK IN ('success','error','timeout')`), `error_message`, `created_at` (unix), `duration_ms`, `tags` (JSON array).

`sessions` (`id TEXT PK`): Ticket groups. Columns: `name`, `created_at`, `ticket_ids_json`.

`flow_definitions` (`name TEXT PK`): Flow metadata. Columns: `description`, `params_schema` (JSON schema), `handler_type` (`builtin`/`proxy`), `target_url`, `enabled`, `tags`.

`flow_logs` (`id INTEGER PK AUTOINCREMENT`): Circular buffer (trigger caps at 1000 rows). Columns: `flow_name`, `ticket_id`, `level`, `message`, `timestamp`.

6.4 Security Posture

1. **No `CommandTemplate`.** Commit `10927ba` eliminated the `command_template` field. All flows are compile-time Go functions—no runtime string interpolation into shell commands.
2. **Method enforcement.** The router checks HTTP method per endpoint (e.g. `POST` only for `/flows/{name}/start`).
3. **XSS mitigations.** The single-file HTML UI (`lean4a_forum.html`) escapes all dynamic content rendered into the DOM. JSON payloads are displayed via `<pre>` with entity-escaped text.
4. **Rate limiting.** Flow semaphore caps concurrent executions at `MAX_CONCURRENT_FLOWS` (default 5). SSE connections time out after 5 minutes.
5. **Local-first privacy.** SQLite lives on the user's filesystem; no data leaves the machine unless the user explicitly exports.

7 Infrastructure Services

All services are orchestrated via `docker-compose.yml` (217 lines). The observability stack (Loki + Grafana) is gated behind a Docker Compose `profile`.

7.1 Service Matrix

Service	Image	Port	Health Check	Depends On
postgres	`postgres:17`	5432	`pg_isready`	—
redis	`redis:7`	6379	`redis-cli ping`	—
nginx	`nginx:stable`	8080	`nginx -t`	app, app_ingestion
app	`Dockerfile.api`	8000	`curl /health/ready`	postgres, redis
app_ingestion	`Dockerfile.ingestion`	8001	`curl /health/ready`	postgres, redis
worker	`Dockerfile.worker`	—	—	postgres, redis
watcher	`Dockerfile.worker`	—	—	postgres, redis
flower	`mher/flower:2.0`	5555	—	redis
web	`frontend/Dockerfile`	3000	`wget :3000/`	nginx
loki†	`grafana/loki:3.4.2`	3100	`wget /ready`	—
grafana†	`grafana/grafana:11.6.0`	3001	`wget /api/health`	loki

† Observability profile only (`docker compose --profile observability up`).

7.2 Nginx Reverse Proxy

Config: `nginx/nginx.conf` (121 lines).
Workers: `auto` (CPU count), 2048 connections.

Rate limiting zones.

```
limit_conn_zone $binary_remote_addr zone=sse_conn:10m;
limit_req_zone  $binary_remote_addr zone=search_req:10m rate=60r/m;
```

Listing 2: Nginx rate-limit zones

Location routing.

Location	Upstream	Rate Limit	Notes
`/api/v1/events/`	`app:8000`	10 conn/IP	SSE: buffering off, 1h timeout
`/api/v1/symbols/search`	`app:8000`	60 req/min +20 burst	Search endpoint
`/admin/`	`app_ingestion:8001`	—	Path rewrite
`/` (default)	`app:8000`	—	All other routes

7.3 Volumes

Volume	Mount Point
`postgres_data`	`/var/lib/postgresql/data`
`redis_data`	`/data` (AOF persistence)
`loki_data`	`/loki`
`grafana_data`	`/var/lib/grafana`
`LEAN_WATCH_ROOT`	`/lean-src:ro` (bind mount)

8 Configuration Reference

All configuration is via environment variables, sourced from `.env` (template: `.env.example`, 38 lines).

8.1 Application & Ports

Variable	Default	Description
`APP_PORT`	8000	Query API listen port
`INGESTION_PORT`	8001	Ingestion API listen port
`NGINX_HTTP_PORT`	8080	Nginx reverse proxy port
`FRONTEND_PORT`	3000	Next.js frontend port
`FLOWER_PORT`	5555	Celery Flower monitoring UI
`LOKI_PORT`	3100	Loki ingester port
`GRAFANA_PORT`	3001	Grafana dashboard port

8.2 Database

Variable	Default	Description
`DATABASE_URL`	(none)	asyncpg DSN
`POSTGRES_DB`	`lean4_automata`	PostgreSQL database name
`POSTGRES_USER`	`lean4`	PostgreSQL user
`POSTGRES_PASSWORD`	`lean4_dev_password`	PostgreSQL password
`POSTGRES_PORT`	5432	PostgreSQL host port

8.3 Redis & Celery

Variable	Default	Description
`REDIS_URL`	`redis://redis:6379`	Redis connection URL
`REDIS_PORT`	6379	Redis host port
`CELERY_BROKER_URL`	`${REDIS_URL}`	Celery broker
`CELERY_RESULT_BACKEND`	`${REDIS_URL}`	Celery result backend

8.4 JWT Authentication

Variable	Default	Description
`JWT_SECRET_KEY`	`change-me-in-production`	HS256 signing key
`JWT_ALGORITHM`	`HS256`	JWT algorithm

8.5 Runtime

Variable	Default	Description
`ENV`	`development`	Deployment environment

ALLOWED_ORIGINS	http://localhost:3000	CORS origins (comma-separated)
NEXT_PUBLIC_API_BASE_URL	http://localhost:8080	Frontend API URL

8.6 Parser & Watcher

Variable	Default	Description
LEAN_WATCH_ROOT	./lean-src	Root directory for .lean file watching
LEAN_PROJECT_ID	1	Project ID used by watcher indexing

8.7 Go Forum Harness

Variable	Default	Description
LEAN4A_FORUM_DB	~/.lean4a/forum.db	SQLite forum DB path
LEAN_DB_URL	http://localhost:8000	Lean4-Automata API URL
TPH_SERVER_URL	http://localhost:5000	TPH GPU server URL
FLOW_TIMEOUT_SEC	30	Flow execution timeout
MAX_CONCURRENT_FLOWS	5	Max parallel flows

8.8 Grafana

Variable	Default	Description
GRAFANA_ADMIN_USER	admin	Grafana admin username
GRAFANA_ADMIN_PASSWORD	admin	Grafana admin password
GRAFANA_ANONYMOUS_ENABLED	false	Anonymous access toggle

9 Observability Stack

The observability stack is **opt-in**, gated behind a Docker Compose profile:

```
docker compose --profile observability up
```

Listing 3: Enabling the observability stack

9.1 Loki (Log Aggregation)

Parameter	Value
Image	`grafana/loki:3.4.2`
Port	3100
Health check	`wget --spider /ready`
Storage backend (dev)	Filesystem (chunks) + in-memory (index)
Log retention (dev)	∼24 hours
Volume	`loki_data:/loki`
Config	`observability/loki/loki-config.yaml`

Loki receives structured JSON logs from all Python and Go services. In production, the storage backend should be switched to an S3-compatible object store.

9.2 Grafana (Dashboards)

Parameter	Value
Image	`grafana/grafana:11.6.0`
Port	3001
Health check	`wget --spider /api/health`
Volume	`grafana_data:/var/lib/grafana`
Default credentials	`admin` / `admin` (override via `.env`)
Anonymous access	Disabled
Sign-up	Disabled

Auto-provisioned resources.

- **Data source:** Loki at `http://loki:3100` (provisioned via `observability/grafana/provisioning/`).
- **Dashboard:** `logs-overview.json` — log throughput, error rates, per-service breakdown (set as home dashboard).

9.3 Celery Flower

Celery Flower (`mher/flower:2.0`, port 5555) provides a web UI for monitoring Celery task queues, worker status, and task results. It connects to Redis via `CELERY_BROKER_URL`.

9.4 Application-Level Metrics

The Go forum harness exposes in-memory metrics at `GET /metrics`:

- Flow execution count (by flow name)
- Flow success/error rate
- Average execution duration

- Active SSE connections
- Ticket creation rate

10 Future Work

The following items are designed but not yet implemented as of commit `10927ba`.

10.1 Not-Yet-Implemented Flows

Four flow entries exist in the registry as placeholders returning `{"status":"not_implemented"}`:

1. **`formula_match`** — Match a DynamicFlux Python formula against Lean symbols. Intended to bridge the numerical Python world with the formal proof world by finding Lean theorems whose signatures structurally correspond to a given formula.
2. **`wolfram_verify`** — Verify a mathematical formula via the Wolfram Engine (invoked through the C# UniversalLLMOrchestrator).
3. **`lean_diff_mathlib`** — Compare local Lean files against upstream Mathlib, identifying declarations that have drifted from or are missing in the canonical library. Requires `lake` integration.
4. **`lean_mathlib_diff`** — Deprecated alias for the above; will be removed in a future commit.

10.2 CLI Wizard

A planned interactive command-line wizard (`lean4a wizard`) that guides users through project setup:

- Detect Lean 4 / `lakefile.lean` presence
- Create project entry in PostgreSQL
- Run initial full-scan indexing
- Display proof-status summary

10.3 Micro-Agent Harness

An extension of the forum harness where AI agents (TPH, DynamicFlux) can autonomously chain flows:

- **Flow chaining:** Output of one flow feeds as input to the next
- **Decision nodes:** Agent inspects intermediate results and branches
- **QMCP consent gate:** Human-in-the-loop approval before executing destructive or expensive flows

10.4 Formula-to-Lean Matcher

A dedicated service that:

1. Parses a Python/DynamicFlux formula AST
2. Normalises it into a type-theoretic signature sketch
3. Searches the Lean symbol index for structurally compatible theorems
4. Returns ranked matches with confidence scores

10.5 Additional Planned Enhancements

- **Export formats:** PDF, Markdown, and JSON export of ticket collections.
- **Session collaboration:** Encrypted shareable links for sessions.
- **AI proof suggestions:** LLM-generated proof-strategy hints based on sorry locations and dependency graphs.
- **Formula library:** Searchable index of DynamicFlux formulas with Lean correspondence annotations.

11 Commit History

The five most recent commits leading to the anchor point:

SHA	Title	What Changed
`10927ba`	refactor: eliminate CommandTemplate code-exec surface—compile-time FlowFunc registry	Removed `command_template` field from `flow_definitions`; all flows are now compiled Go functions. Eliminates shell-injection surface entirely.
`4a3c84a`	security: audit pass—fix XSS, proxy DoS, JWT leakage, SSE method mismatch	HTML entity escaping in forum UI; proxy response-size cap; JWT secret rotation docs; SSE endpoint enforces GET-only.
`855b7b9`	feat: forum-style UI + Go_Red CLI harness	Added `go/forum.go`, `go/forum_schema.sql`, `ui/lean4a_forum.html`, `ui/flow_definitions.json`, and `docs/forum/ARCHITECTURE.md`.
`c40a94a`	feat: full stack—ingestion service, parser enrichment, static-analysis metrics	Migration 0004 (enriched metrics), `FileMetrics` dataclass, ingestion API, 3 new views (`v_proof_status`, `v_floating_symbols`, `v_file_interconnect`).
`07fc8b3`	Add optional Grafana Loki log aggregation	`docker-compose.logging.yml`, `observability/` directory with Loki config, Grafana provisioning, and dashboards.

Earlier commits established the infrastructure skeleton (`b92878e`), parser + Celery workers (`07752ca`), query API + SSE (`f9cd528`), and Next.js frontend (`50bf169`).